

Contributions:

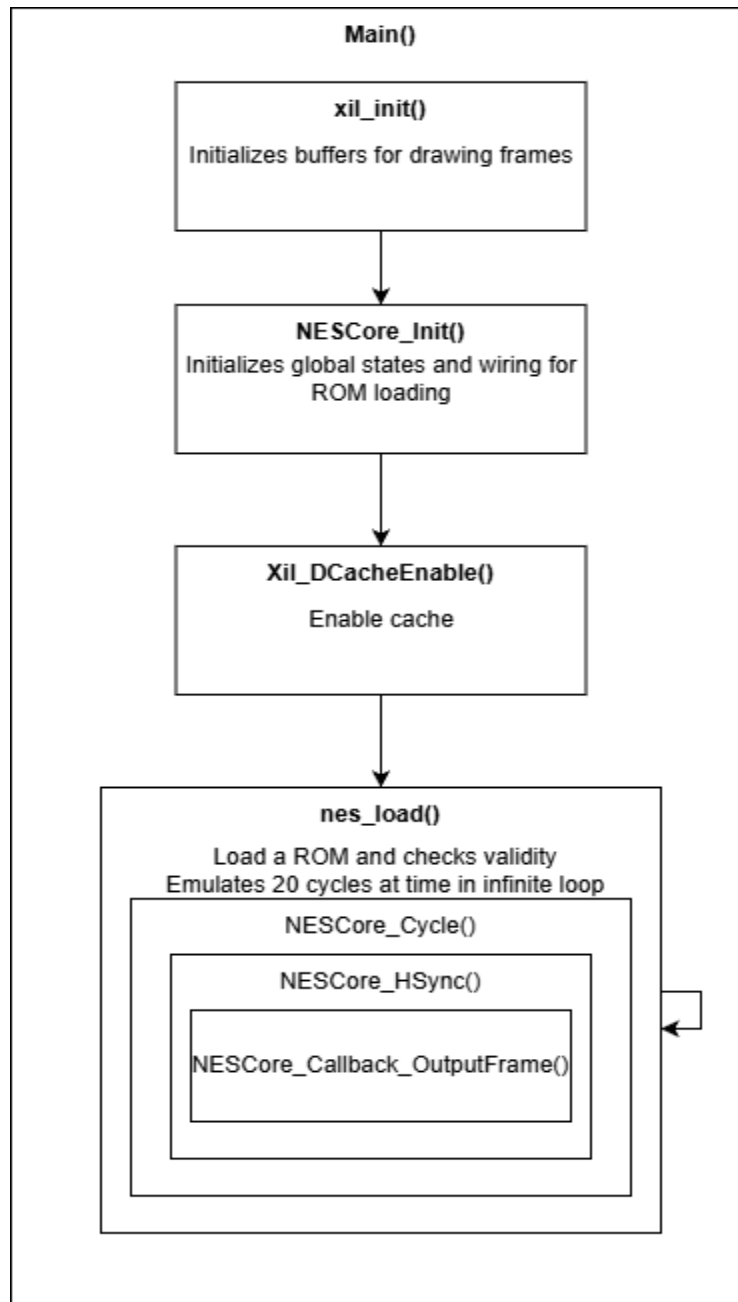
Caleb Hemmestad (Group A) - 25%

Wilson Diep (Group A) - 25%

Penny Mayer (Group B) - 25%

Sandee Madan (Group B) - 25%

In your report, describe how nes_bootloader.c currently works. Using a similar approach as what is presented in Chapter 1 of the Wolf textbook, draw a high-level structural diagram. How does NESCore_Callback_OutputFrame() get called?



Click three different green boxes and in your writeup, describe what configuration options are available and how they may be potentially useful in an embedded system.

Clock Generation

There appear to be configurations for Processor/Memory clocks, which are presumably internal to the Processing System. There are also four external clocks, which can be set independently and routed to peripheral IPs.

High Performance AXI Slave Ports

The Processing System appears to support up to four “high performance” AXI slave interfaces, at either 32 or 64 bit data widths. As slave-side ports, they can be connected to a peripheral which can then request specific memory from the PS.

I/O MUX (MIO)

In the I/O MUX configuration, it gives you the option to select and deselect peripherals. It also has a sort of block diagram for selecting and deselecting certain parts of the peripherals that you want to use.

For the Block Design you just created: Are these buttons, LEDs, and switches connected via the PS subsystem or the PL subsystem? Briefly defend your answer. Note also that all three peripherals appear to be the same exact IP type (axi_gpio) – how can this be possible?

The peripherals are connected via the PL subsystem. In our diagram, the interconnect between the PS and PL is the AXI Interconnect IP. This IP interfaces the multiple AXI GPIO IP which are configurable to receive and send I/O from different peripherals, such as the LEDs, buttons, and switches. As long as the peripherals are compatible with the AXI GPIO, it doesn't matter which one it is, they can all utilize the same IP type.

The documentation for the axi_gpio is available in the docs/IP/ directory. Based on the datasheet and the address map shown in the “Address Editor” (mentioned in instruction 7 of Step 2: Use Designer Assistance), how would you (in software) read the current state of the switches? Be specific.

If you wanted to access the current state of the switches, you would need to read from the S_AXI_* wires particularly the 32 bit s_axi_data wire.

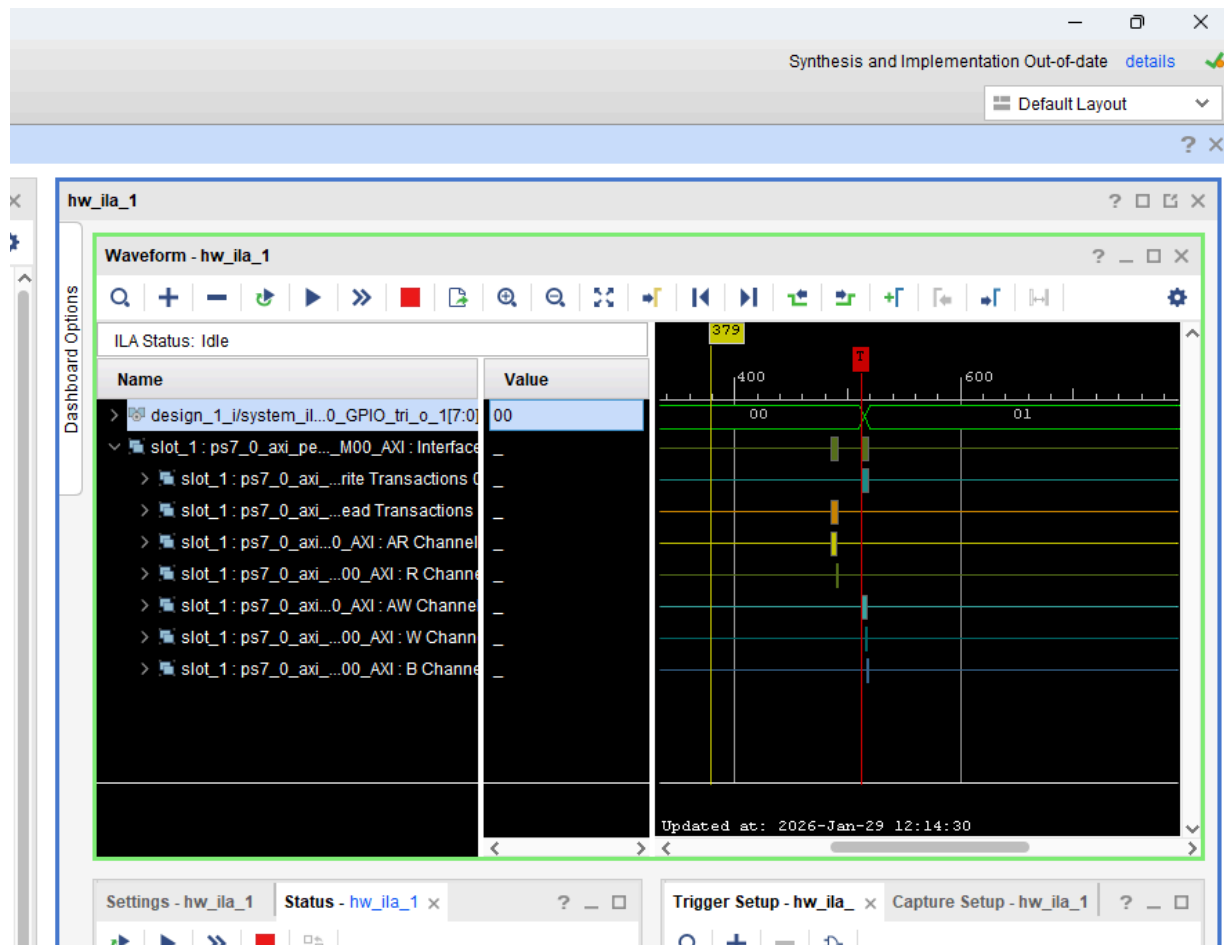
Something like

```
unsigned volatile int data = s_axi_data & 0xFFFFFFFF;
```

A nice feature is the ability to right-click on any function to view its declaration – do this for the print() function. In your writeup, use this feature and describe what print() does, and how. Why do you believe this function is used by Xilinx for their Hello World application, as opposed to the more conventional printf() function?

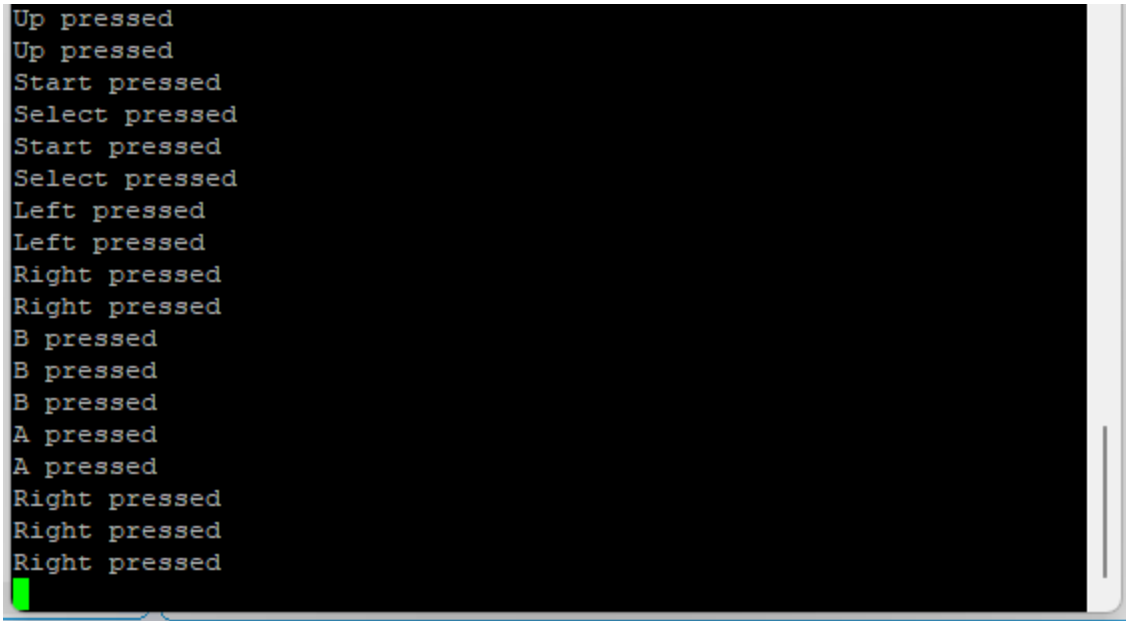
print() works by taking in a const char pointer. It continually prints out one char at a time, calling the function outbyte and keeps printing over uart until the value at the pointer's address is null.

Take a screen capture of an LED wire turning on? Can you turn the LED on and off fast enough to get a screen capture of the Logic Analyzer displaying this pulse? If so, then provide this screen capture as well. For how long does the pulse stay high?



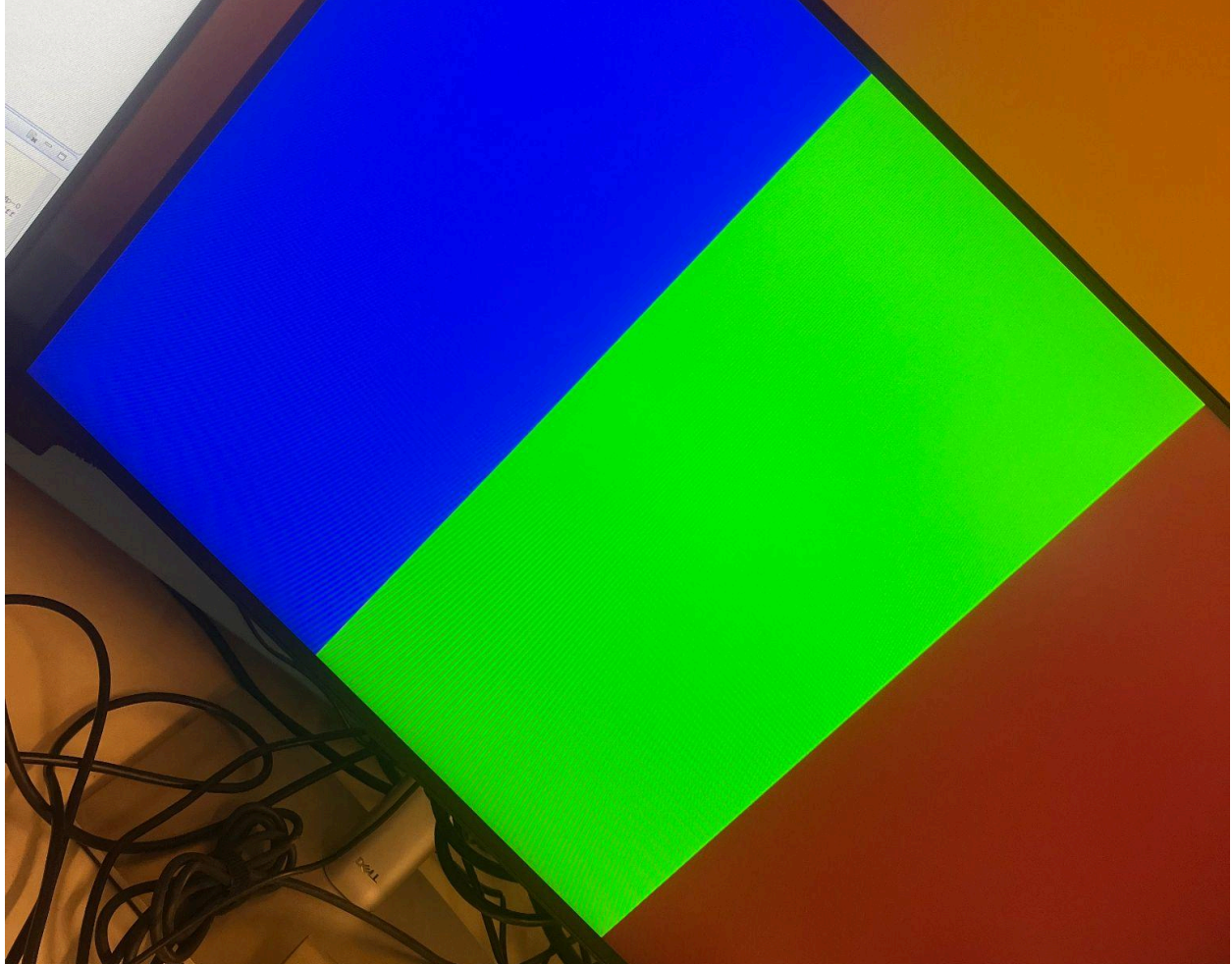
In this waveform, we were not able to press the button fast enough to determine how long the signal was high for. The wave form keeps getting cut off at 1024 data depth. We tried to increase the max data depth of the ILA core but it throws an error saying the max depth is 1024.

Modify the hello_world application to also interface with the switches, buttons, and LEDs that are configured in the programmable logic. For example, have the application print out the state of the switches when a button is pressed, or light up certain LEDs given an input integer.



```
Up pressed
Up pressed
Start pressed
Select pressed
Start pressed
Select pressed
Left pressed
Left pressed
Right pressed
Right pressed
B pressed
B pressed
B pressed
A pressed
A pressed
Right pressed
Right pressed
Right pressed
```

In VIVADO, add these peripherals to your project , connect and then configure them to generate a 640x480 output signal



Modify the configuration registers for correct VDMA operation, and in your writeup, provide a justification based on the VDMA documentation for how you set these values.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_CR_OFFSET, 0x03);
```

This starts the mm2s and activates the circular park.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_HI_FRMBUF_OFFSET, 0x00);
```

This selects the frame buffer being used.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_MM2S_ADDR_OFFSET + XAXIVDMA_START_ADDR_OFFSET, test_image);
```

This is the start address of what we want to buffer and display.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_MM2S_ADDR_OFFSET + XAXIVDMA_STRD_FRMDLY_OFFSET, 0x0500);
```

This sets the stride size which is 1280 bytes.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_MM2S_ADDR_OFFSET  
+ XAXIVDMA_HSIZE_OFFSET, 0x0500);
```

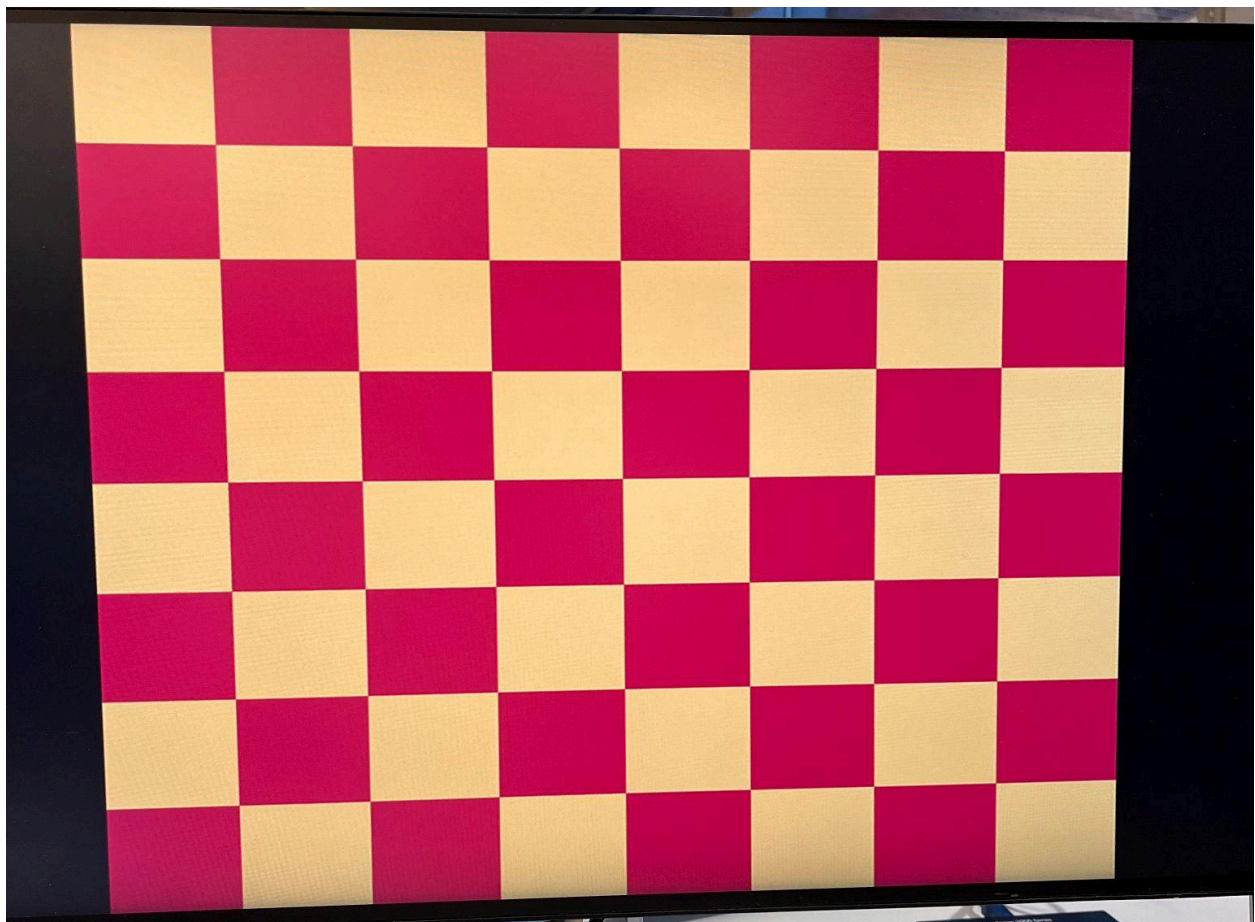
This sets the horizontal size which is 1280 because there are 640 pixels per line and each pixel is 16 bits.

```
XAxiVdma_WriteReg(XPAR_AXI_VDMA_0_BASEADDR, XAXIVDMA_MM2S_ADDR_OFFSET  
+ XAXIVDMA_VSIZE_OFFSET, 0x01E0);
```

This sets the vertical number of lines which is 480.

In your writeup, explain how you converted these color values for the 16-bit framebuffer.

I simply just took the first 4 bits from the 16 bit color segment and matched the order for how we have it set in hardware. Example: RGB hex code for Cyclone Cardinal is 0xC8103E so what we would get for our 16 bit color is 0x0C13. Since the order for our bits for connecting the colors are different from the order given, what we used is 0x031C.



Modify the nes_bootloader code such that the NES games are reasonably playable. In your report, describe your general approach to implementing both of the NESCore_Callback functions.

For the nes_bootloader code, we just add the variables used for the test_image except we changed the start address for buffering the image to FBUFFER_BASEADDR. For implementing the NESCore_Callback functions, we added to the InputPadState to set the buttons to work with the emulator, and for the OutputFrame we had each pixel given output as twice the size to fill the entire vertical space and then added black bars to fill the empty space on the sides to make sure the coloring is correct.

BONUS credit. MP-0's bonus point criteria is creativity. The current nes_bootloader design is very limited in terms of functionality.

For bonus features, team SA-4 Group A implemented a text-based menu selection using Putty.